

面向 ZU4EV（开发板目标平台：本项目最后上板验证的实车跑道）的资源感知软判决维特比译码器

0. 摘要与术语首现说明

本项目实现了一个面向课程开放项目的 Viterbi Decoder（维特比译码器：像在时间展开的岔路地图里找最可能路线的硬件侦探）。目标不是只写一个能跑的算法，而是从 Python model（Python 模型：像先在纸上算一遍标准答案的老师版解法）、test vector（测试向量：统一发给软件、仿真和板子的考题）、RTL（寄存器传输级：把硬件写成寄存器和组合逻辑之间搬数据的蓝图）、Vivado synthesis（Vivado 综合：把 RTL 翻译成 FPGA（现场可编程门阵列：像能反复重搭的数字积木板）资源网络的编译步骤）、Vivado implementation（Vivado 实现：把资源放到芯片坐标并连线的布线步骤）、board validation（板级验证：把设计放到真实板子上跑，像论文结果的路考）一直闭环到最终报告。

下面的表是本报告第一次出现的核心英文术语说明。后文再次出现这些词时不重复解释；文件名、路径、命令、代码字段保持英文。

English term	中文解释和比喻
Resource-Aware	资源感知：像做预算一样同时盯住资源、功耗和时序
Soft-Decision	软判决：不是说 0/1，而是告诉译码器我有多确定，像天气预报给概率
Hard-Decision	硬判决：只给 0/1，像只说对或错不说信心
VLSI	超大规模集成电路：把算法压进芯片面积、时钟和功耗约束里的工程世界
LUT	查找表：FPGA 里实现组合逻辑的小真值表，像可改写的小算盘
FF	触发器：每个时钟保存一位状态的小抽屉
Convolutional Code	卷积码：把当前 bit 和历史 bit 混合产生冗余，像带记忆的打包机
Trellis	网格图：把每个时刻的状态和岔路摊开成地图
Branch Metric	分支度量：一小段路和收到数据有多不像
Path Metric	路径度量：整条候选路线累计的不像程度
ACS	加比选单元：先加成本、再比较、最后留下赢家的路口裁判
Traceback	回溯：从终点倒着沿赢家箭头找回原始路线
Subtract-Min Normalization	减最小值归一化：所有路线成本一起减掉最低成本，像大家同时把海拔零点下移
BMU	分支度量单元：专门给每条小路打分的小秤
Survivor RAM	幸存路径存储器：记录每个路口谁赢了的本本
ARM	嵌入式处理器核：Zynq 里负责控制流程的小电脑
PS	处理系统：Zynq 里的 ARM 处理器部分，像板上的司机
PL	可编程逻辑：Zynq 里的可重搭硬件区域，像可重搭的硬件车间
AXI DMA	AXI 直接内存访问：PS 和 PL 之间搬数据的快递员
UART	串口：板子把日志打印给电脑的嘴
Bitstream	硬件配置流：告诉 FPGA 怎样搭电路的施工图
ELF	可执行程序文件：ARM 要运行的裸机程序包
JTAG	调试下载接口：电脑临时把硬件配置流和可执行程序文件放进板子的线缆通道
OCM	片上存储器：Zynq 内部的小而快的共享储物柜
WNS	最差负裕量：时序最危险路径还差或多出多少时间
TNS	总负裕量：所有时序欠账加起来的总额
BRAM	块存储器：FPGA 内部成块的储物柜
DSP	数字信号处理硬核：FPGA 里专门做乘加的工具台
Power	功耗：电路运行消耗的电力预算

English term	中文解释和比喻
BER	误码率：错 bit 占总 bit 的比例，本报告只用有限向量不匹配率做谨慎近似
AWGN	加性白高斯噪声：像均匀洒在信号上的细小雨点
Case	测试用例：一套输入、期望输出和元数据组成的一张考卷
Mismatch	不匹配位：译码输出和标准答案不同的一位，像改卷时圈出的错题
Run ID	运行编号：给每次实验贴上的时间戳小票
Hero Design	主打设计：最终重点优化和上板验证的版本，像参赛主力车
Board Shell	板级外壳：包在核心外面的 PS、DMA、时钟和日志系统
DDR	外部动态内存：容量大但路径更长的板外仓库
Pipeline/Retiming	流水线/重定时：把长组合路径拆成多段接力
SNR	信噪比：信号强度和噪声强度的比值，像说话声比背景声大多少
AI tools	AI 工具：辅助规划、写脚本和整理报告的工程助手，不替代验证责任

结论先说清楚：功能闭环通过。Python 单元测试 6/6 通过；硬判决 RTL 仿真 4 个必需 case 全 0 mismatch；软判决 RTL 仿真 3 个 AWGN case 全 0 mismatch；M7 最新板级 run_id 为 `m7\_board\_20260502\_083547`，UART 显示 3 个 case 全部 0 mismatch。限制也必须先说清楚：M6 在 100 MHz 目标下 hero design 时序未收敛，布局布线后的 WNS 为 -20.433 ns；M7 board shell 降到 25 MHz 后通过，说明当前架构功能正确但还不是 100 MHz timing-clean design（时序干净设计：所有路径都能在目标时钟内到达的硬件版本）。

1. 引言与项目动机

通信和存储系统都要面对噪声。重传不是总能解决问题：深空链路、低功耗无线节点、实时视频链路、甚至板上高速数据通道，都可能没有足够时间或带宽反复重来。纠错码的价值就是把冗余提前放进数据里，让接收端能在不重传的情况下修正一部分错误。

本项目选择维特比译码器，是因为它同时有清楚的算法结构和清楚的硬件压力。算法上，它是在 trellis 中寻找最低 path metric 的路径；硬件上，它不断重复 BMU、ACS、survivor 写入和 traceback。这个结构非常适合 VLSI 课程关注的 algorithm/architecture co-design（算法/架构协同设计：算法规则和硬件结构一起取舍，像同时设计路线和车辆）。

成熟算法仍有研究价值，因为真正难点不在“能不能译码”，而在“怎样在给定资源、频率、精度和板级接口里译码”。同一个算法可以有硬判决、软判决、不同 traceback depth、不同 path metric width、不同 normalization 方案。这些选择直接改变正确率、LUT/FF 占用、时序和功耗。

2. 问题定义与设计规格

设计参数唯一事实源是 `spec/viterbi\_spec.json`。本项目的正式配置是 rate-1/2（码率二分之一：每 1 个原始 bit 生成 2 个编码 bit）、K=7（约束长度 7：编码器记住当前 bit 和前 6 个历史 bit）、[171,133]（八进制生成多项式：两条 XOR 抽头规则，像两把不同齿形的梳子梳过 shift register）。因为 K=7，所以状态数是 $2^{(K-1)}=64$ 。

项目项	取值	数据来源	为什么这样选
编码	Convolutional Code rate 1/2, K=7, [171,133]	spec/viterbi_spec.json	经典卷积码配置，复杂度足够体现 64-state VLSI 架构压力
baseline	Hard-Decision	spec/viterbi_spec.json	先建立 0/1 输入保底版本，便于定位 trellis 和 state numbering
hero	3-bit Soft-Decision, depth 40, PM width 12, subtract-min	spec/viterbi_spec.json	软信息提升鲁棒性，depth 40 在 sweep 中当前向量全 0 mismatch
payload	96 bit	vectors/*/metadata.json	足够覆盖一帧，同时让 RTL/board regression 快速迭代

项目项	取值	数据来源	为什么这样选
board	MZU04A-4EV / XCZU4EV	spec/viterbi_spec.json; config/local.env	用户现有 ZU4EV 板卡和 Xilinx 2024.1 工具链

3. 从零理解 Viterbi 译码

可以把译码想象成在雨夜开车。编码器发出的每两个 coded bits（编码 bit：加了冗余后的输出 bit）像路标，但噪声会把路标弄脏。接收端不知道哪条路是真的，只能把所有可能路径摊开比较。

Trellis 是这张时间展开的岔路地图。每个时刻有 64 个状态，每个状态有两条可能的下一步，对应输入 bit 为 0 或 1。Branch Metric 衡量当前收到的符号和某条小路预期符号的差距。Path Metric 把一路走来的差距累加起来。ACS 在每个状态入口只保留更便宜的路径。Survivor RAM 记录赢家来自哪里。Traceback 最后从终态倒着找回输入 bit。

硬判决时，收到的是 0 或 1，branch metric 基本就是汉明距离。软判决时，收到的是 0 到 7 的置信度；越靠近理想值，成本越低。3-bit soft symbol（3 位软符号：0 到 7 的置信度刻度，像 8 档温度计）让译码器知道“这个 1 很像 1”还是“这个 1 其实有点可疑”。

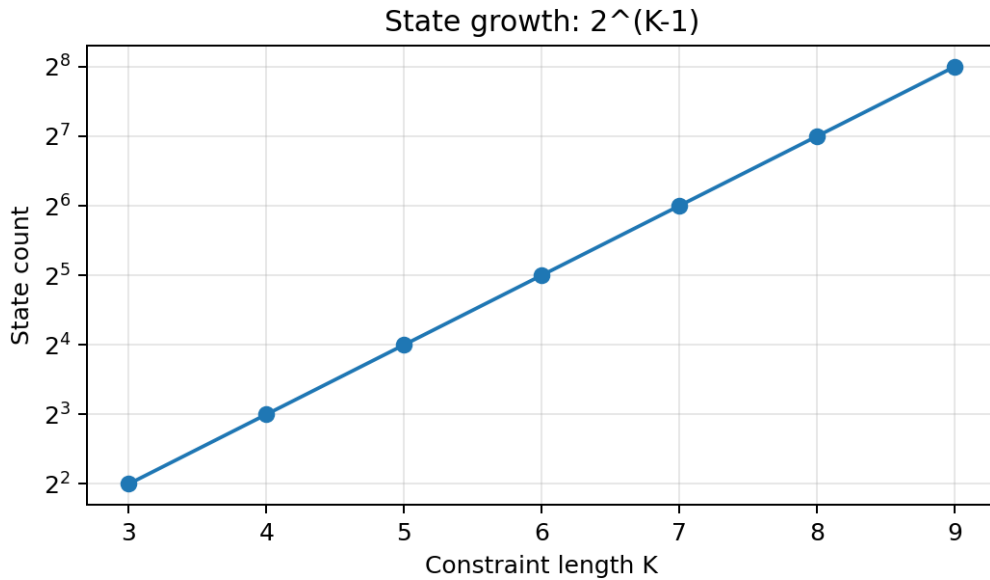


图 4：状态数随 K 指数增长。 这张图来自 `scripts/build\_final\_report.py` 的公式 $2^{(K-1)}$ ，回答为什么 K=7 会直接变成 64 个状态。它不是实验结果图，而是设计复杂度图。

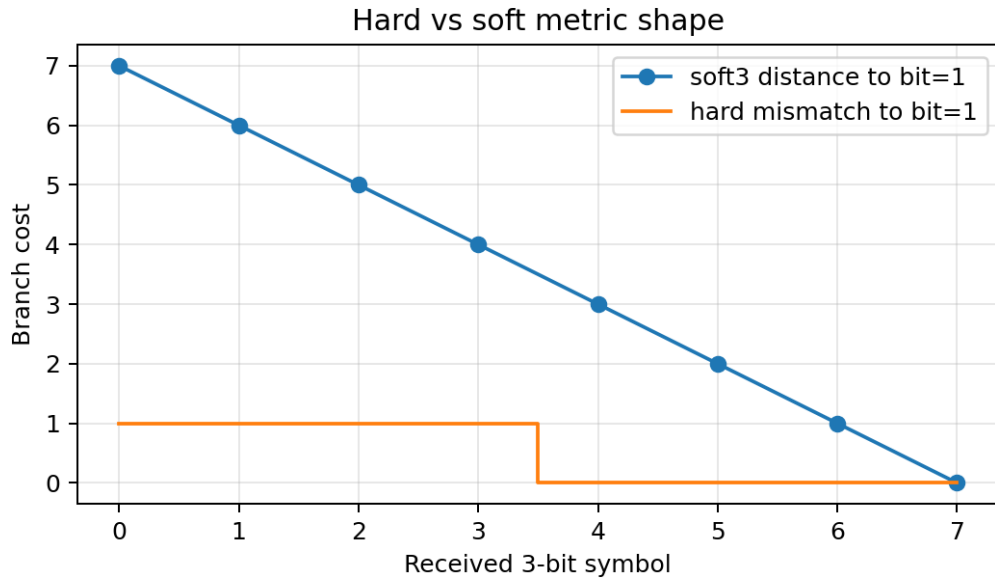


图 5：硬判决和软判决分支成本形状。 这张图展示 hard metric 是台阶，而 soft3 metric 有细粒度距离。图来自 `scripts/build\_final\_report.py`，用于解释为什么 soft input 能保留更多信息。

4. 仓库结构与事实源

本仓库按 phase 留证据，不靠口头结果。关键目录如下：

路径	角色
spec/viterbi_spec.json	设计参数事实源，RTL package 和向量生成都从这里读取
model/	Python encoder、channel 和 golden decoder
vectors/	M2 生成的统一输入输出题目，每个 case 有 metadata.json
rtl/viterbi_core/	hard baseline 与 soft3 hero core
rtl/system/	M7 PS+DMA+PL board shell wrapper
tb/	xsim testbench
vivado/tcl/	综合、实现和板级 block design 脚本
vitis/zu4ev_baremetal/	裸机 DMA/UART board harness
data/	模型、仿真、sweep、Vivado、board 原始结果
docs/assets/	Mermaid 源图和 Python 生成图
WORKLOG.md / DECISIONS.md / EXPERIMENTS.yaml	执行记录、决策记录、实验索引

数据来源严格分层：`data/model/model\_unit\_test\_summary.json` 记录 Python 单元测试；`data/regression/\*.csv` 记录 RTL 仿真；`data/analysis/sweep\_results.csv` 记录参数扫描原始行；`data/impl/vivado\_summary.csv` 记录 Vivado 资源/时序/功耗；`data/board\_runs/board\_summary.csv` 记录 UART run。

5. 端到端工程流程

```

flowchart LR
    SPEC["spec/viterbi_spec.json"] --> PY["Python golden model"]
    PY --> VEC["vectors with metadata"]
    VEC --> RTL["RTL core and testbench"]
    RTL --> XSIM["xsim bit-true regression"]
    RTL --> VIV["Vivado synthesis and implementation"]
    VIV --> XSA["XSA and bitstream"]
    XSA --> BOARD["ZU4EV JTAG plus UART run"]
    BOARD --> REPORT["Report.md and Report.pdf"]
  
```

图 1：端到端工程闭环。 Mermaid 源思想对应
`docs/assets/diagrams/end\_to\_end\_workflow.mmd`，这里直接嵌入报告，回答每个 artifact
如何从上一步产生，而不是孤立手写。

6. 卷积码与 trellis 构造

卷积编码器内部是 shift register（移位寄存器：像一排会向前挪的记忆格子）。K=7 表示寄存器宽度为 7，其中 1
个是当前输入，6 个是历史。每输入一个 bit，就按 [171,133] 两个多项式各 XOR 一次，产生两个输出 bit。

状态只需要记住历史 6 个 bit，所以状态数是 $2^6=64$ 。`model/trellis.py` 和 `rtl/common/viterbi\_pkg.sv` 使用同一套 next_state 和
expected_output 规则，这避免了 Python 与 RTL 状态编号不一致。M1 的 K=3 smoke test 先用 4-state 小地图验证逻辑，M1
正式测试再切到 K=7。

```
flowchart LR
    in["input bit"] --> sr["7-bit shift register"]
    sr --> g0["poly 171 XOR"]
    sr --> g1["poly 133 XOR"]
    g0 --> out0["encoded bit 0"]
    g1 --> out1["encoded bit 1"]
```

图 2：K=7 编码器抽头示意。 源图对应 `docs/assets/diagrams/conv\_encoder\_k7.mmd`，回答 [171,133]
为什么不是神秘数字，而是 XOR 连接规则。

7. 分支度量、软判决与定点路径度量

硬判决 baseline 的 branch metric 是 0、1、2 三种距离。soft3 hero 的 branch metric 把每个 coded bit 的理想 0 映射到 0，理想 1
映射到 7，再求绝对差。两个输出 bit 的差相加，就是当前 trellis branch 的成本。

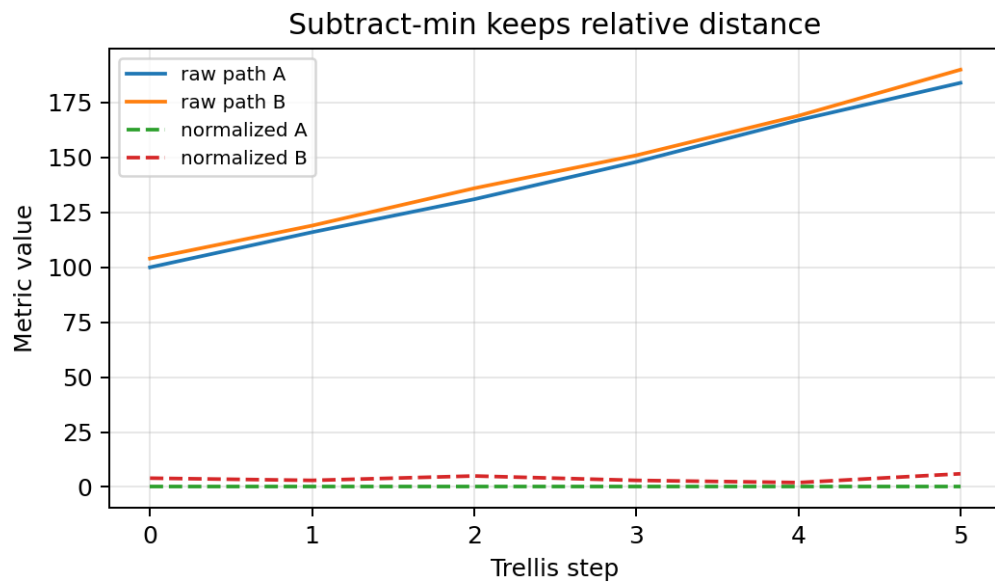
Path metric 会不断增长，所以硬件不能无限宽。M5 扫描 path metric width = 8/10/12/16，normalization =
none/subtract_min。当前向量集下，hero 默认 depth 40、PM width 12、subtract-min 全 0 mismatch；非零 mismatch 只出现在
soft_awgn_0db 且 traceback depth 16，共 64 个 mismatch，first mismatch 为 3。

traceback_depth	payload_bits_total	mismatch_count_total	observed_mismatch_rate
16	864	8	0.009259
32	864	0	0.000000
40	864	0	0.000000
64	864	0	0.000000

表：traceback depth sweep 汇总。 数据来源 `data/analysis/traceback\_depth\_sweep.csv`，由 M5 原始
`data/analysis/sweep\_results.csv` 派生。它回答短回溯是否会损失正确性。

path_metric_width	payload_bits_total	mismatch_count_total	observed_mismatch_rate
8	864	0	0.000000
10	864	0	0.000000
12	864	0	0.000000
16	864	0	0.000000

表：path metric width sweep 汇总。 数据来源 `data/analysis/path\_metric\_width\_sweep.csv`。它回答当前向量集下 8/10/12/16
bit 是否改变 mismatch。结果显示 depth 40 时未观察到 mismatch
差异；这不是证明所有信道都无差异，只说明当前测试集如此。



**图 9：subtract-min normalization 概念图。 ** 这张图来自 `scripts/build\_final\_report.py` 的确定性示例，回答 normalization 为什么不改变相对赢家，却能压低数值范围。

8. RTL 架构

硬件分为 kernel scope（核心范围：只看译码算法本身，像发动机）和 board-shell scope（板级外壳范围：DMA、寄存器、PS/PL 连接，像底盘和仪表）。核心模块包括 `bmu\_hard.sv`、`bmu\_soft3.sv`、`acs\_unit.sv`、`acs\_array.sv`、`path\_metric\_bank.sv`、`path\_metric\_normalizer.sv`、`survivor\_ram.sv`、`traceback\_engine.sv` 和 `viterbi\_decoder\_core\_soft3.sv`。

```
flowchart LR  
  RX["rx symbols"] --> BMU["BMU"]  
  BMU --> ACS["ACS array"]  
  ACS --> PM["path metric bank"]  
  PM --> SURV["survivor memory"]  
  SURV --> TB["traceback"]  
  TB --> OUT["decoded bits"]
```

**图 10：RTL core architecture。 ** 源图对应 `docs/assets/diagrams/viterbi\_core\_architecture.mmd`，回答数据如何从输入符号走到最终 decoded bit。

M7 新增 board shell：`viterbi\_axis\_wrapper.sv` 把 soft3 core 接成 AXI Stream；`viterbi\_control\_regs.v` 提供 AXI-Lite 控制寄存器；`viterbi\_zu4ev\_shell.v` 作为 Vivado block design module reference。控制寄存器提供 `sample\_count`、`status`、`cycle\_count`、debug seen/emitted counters。这个设计让 board app 能用同一套 vector 比对 golden output。

9. 验证方法

Waveform（波形：每根信号随时间变化的显微镜）只适合 debug，不能作为最终 PASS/FAIL。最终判断必须来自 bit-true comparison（逐 bit 精确比较：每个输出 bit 都和 golden 文件对齐检查）。因此本项目每层都复用同一事实源：

层级	输入	判定	结果文件
Python	pytest synthetic cases	assert exact bits	data/model/model_unit_test_summary.json
RTL hard	vectors/no_noise 等	xsim output vs golden_decoded.hex	data/regression/rtl_regression_summary.csv
RTL soft3	soft_awgn_0db/1db/2db	xsim output vs golden_decoded.hex	data/regression/soft3_regression_summary.csv
Vivado	RTL top	synthesis/implementation reports	data/impl/vivado_summary.csv
Board	Vitis DMA buffers	UART case lines and mismatch count	data/board_runs/board_summary.csv

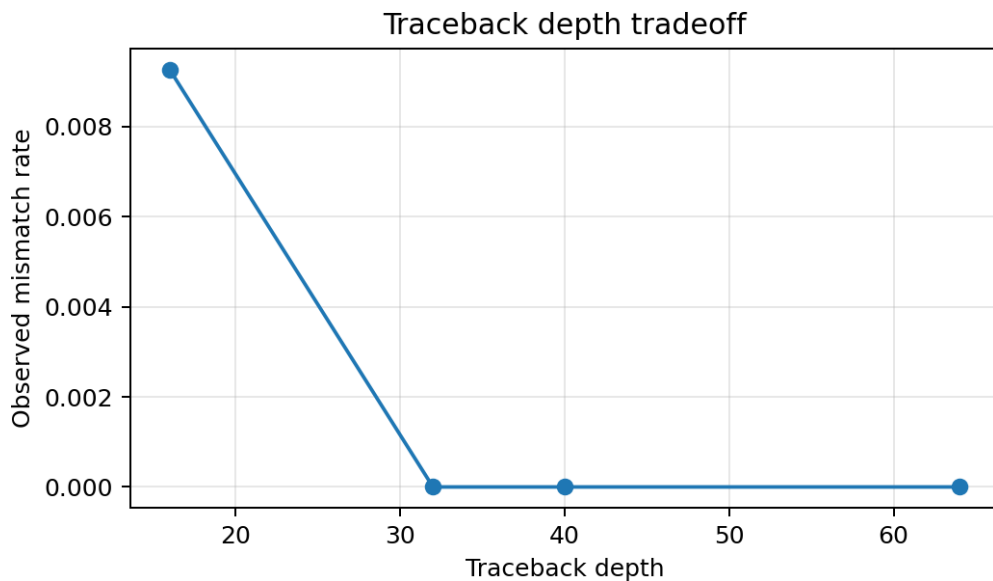
M1 Python pytest 结果：`{'phase': 'M1', 'exitstatus': 0, 'passed': 6, 'failed': 0, 'results': [{'test': 'tests/test_model.py::test_k3_smoke_no_noise_zero_mismatch', 'outcome': 'passed'}, {'test': 'tests/test_model.py::test_k3_smoke_soft3_no_noise_zero_mismatch', 'outcome': 'passed'}, {'test': 'tests/test_model.py::test_k7_formal_no_noise_zero_mismatch', 'outcome': 'passed'}, {'test': 'tests/test_model.py::test_k7_formal_tail_termination_returns_to_zero_state', 'outcome': 'passed'}, {'test': 'tests/test_model.py::test_k7_single_encoded_bit_error_is_corrected_for_small_case', 'outcome': 'passed'}, {'test': 'tests/test_model.py::test_k7_soft3_finite_width_subtract_min_zero_mismatch', 'outcome': 'passed'}]}`。M3 hard RTL 通过 4 个 case。M4 soft3 RTL 通过 3 个 case。M5 sweep 共 288 行，累计 mismatch 64，非零行数 8。

10. 仿真与 BER 分析

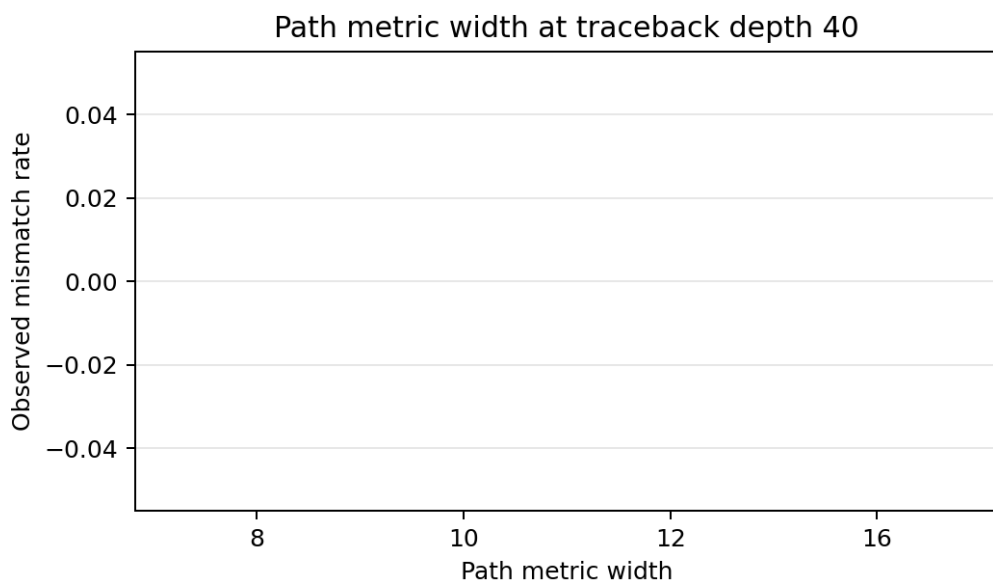
严格说，当前数据不是大样本通信 BER 曲线。每个 case 只有 96 payload bits，所以本报告使用 observed mismatch rate（观测错配率：当前有限向量里错 bit 的比例，像小考错题率）描述结果，不把它夸大成统计充分的 BER。

`data/analysis/ber\_summary.csv` 由 `data/analysis/sweep\_results.csv` 派生，保留每一行的 `run\_id`。最关键现象是：depth 16 在 `soft\_awgn\_0db` 上出现 mismatch，depth 32/40/64 在当前向量集上恢复为 0 mismatch。这个结论支持 hero 选择 depth 40：它比

16 更稳，又比 64 少 traceback latency。



**图 14：traceback depth tradeoff。 **数据来源 `data/analysis/traceback\_depth\_sweep.csv`， run_id 来自 M5 `sweep\_results.csv`。 它回答 depth 变短是否真的会伤正确性。



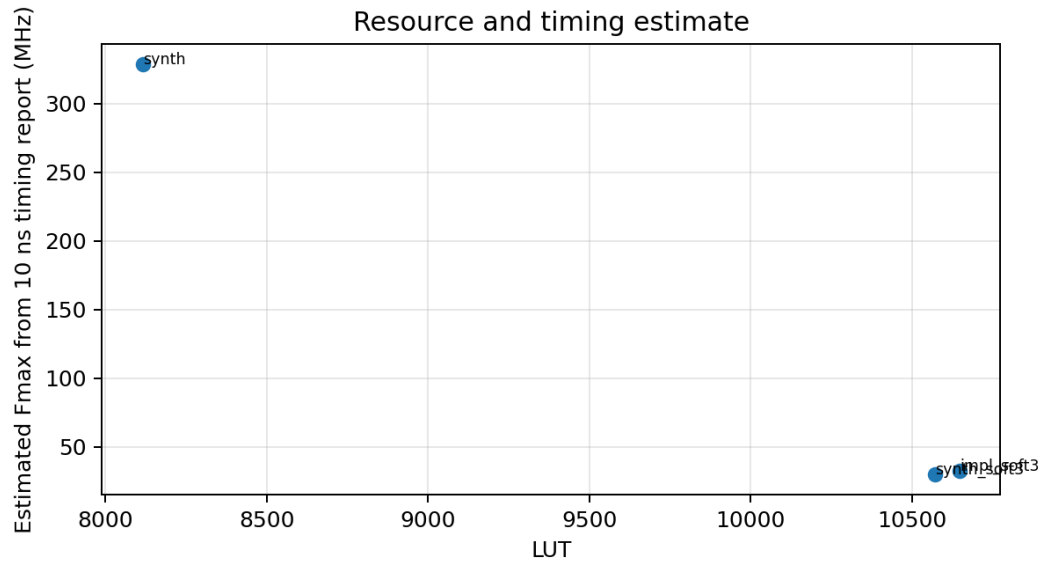
**图 15：path metric width tradeoff。 **数据来源 `data/analysis/path\_metric\_width\_sweep.csv`。 它回答当前测试集在 depth 40 下是否对 PM width 敏感；结果未观察到 mismatch 差异，但资源/时序仍受实现结构影响。

11. 综合与实现结果

run_id	kind	LUT	FF	BRAM	DSP	WNS(ns)	TNS(ns)	Power(W)
m6_synth_viterbi_decoder_core	synth	8116	15243	0	0	6.956	0.000	0.410

run_id	kind	LUT	FF	BRAM	DSP	WNS(ns)	TNS(ns)	Power(W)
m6_synth_viterbi_decoder_core_soft3	synth	10570	17118	0	0	-23.241	-17607.301	0.469
m6_impl_viterbi_decoder_core_soft3	impl	10647	17118	0	0	-20.433	-16272.939	0.487

**表：Vivado resource/timing/power summary。 ** 数据来源 `data/impl/vivado\_summary.csv`。 LUT 是查找表资源， FF 是触发器资源， BRAM 是片上块 RAM， DSP 是乘法硬核， WNS/TNS 是时序裕量， Power 是 Vivado vector-less power report。 这里必须诚实：hero implementation return code 为 0， 但 100 MHz 时序不满足。



**图 16：资源与估算 Fmax。 ** 数据来源 `data/impl/vivado\_summary.csv`， 由 10 ns timing report 的 WNS 估算。 它回答资源增加和最长路径压力如何一起出现。 该估算不是新的 Vivado run， 只是报告数据的后处理。

12. 时序、吞吐率、延迟与功耗分析

吞吐率公式：

$$\text{throughput} = \text{decoded_bits_per_cycle} * \text{clock_frequency}$$

当前 core 每帧输出 96 decoded bits。 M7 board run 记录 `no\_noise` cycles=802, `single\_bit\_error` cycles=855, `soft\_awgn\_2db` cycles=855。 在 25 MHz board shell 下， 按整帧平均吞吐率估算：

$$\text{throughput_no_noise} = 96 / 802 * 25\text{e}6 = 2.99 \text{ Mbit/s}$$

$$\text{throughput_soft_awgn_2db} = 96 / 855 * 25\text{e}6 = 2.81 \text{ Mbit/s}$$

延迟公式：

$$\text{latency} = \text{measured_cycles_to_first_valid_output}$$

本次 board log 记录的是整帧完成 cycles， 不是 first-valid cycle。 因此报告只给整帧 cycles， 不伪造 first-valid latency。 功耗每 bit 公式：

$$\text{energy_per_bit} = \text{power} / \text{throughput}$$

M6 hero implementation power 为 0.487 W，但该 power 来自 100 MHz implementation 的 vector-less 估计；M7 board shell 在 25 MHz 下运行，不能直接把 0.487 W 当作实测板级功耗。若只做同条件粗估，100 MHz 且 855 cycles 的 frame throughput 约 11.23 Mbit/s，对应 energy_per_bit 约 43.4 nJ/bit。这个数字必须标注为 Vivado 估算，不是板上功耗表读数。

13. 板级系统设计与硬件测试

M7 board shell 使用 PS、PL、AXI DMA、UART、JTAG 和 Vitis bare-metal（Vitis 裸机程序：不跑操作系统，直接控制 DMA 和打印 UART 的 ARM 程序）。数据流是：Vitis app 把 `rx\_soft3.hex` 生成的 16-bit buffer 放入 OCM，AXI DMA 的 MM2S 通道把输入送到 PL，Viterbi core 输出 96 个 decoded bits，S2MM 通道写回 OCM，ARM 再和 golden array 比较。

```
flowchart LR<br/> APP["Vitis app in PS"] --> DMA["AXI DMA"]<br/> DMA --> PLIN["MM2S stream"]<br/> PLIN --> CORE["Viterbi soft3 core in PL"]<br/> CORE --> PLOUT["S2MM stream"]<br/> PLOUT --> DMA<br/> APP --> UART["UART log"]<br/> JTAG["JTAG"] --> APP<br/> JTAG --> CORE
```

**图 12：PS+DMA+PL+UART board shell。 ** 源图对应 `docs/assets/diagrams/ps\_pl\_board\_shell.mmd`。它回答 kernel scope 如何被板级 I/O 包起来。

关键 debug 结论是 OCM 地址段。第一次 DDR buffer run 没有 stream input。第二次 OCM buffer run 收到 109 个 sample 后 DMA decode error。对比 FIR 参考 shell 后发现 block design 自动把 `SEG\_ps\_0\_HPC0\_LPS\_OCM` excluded。补上 `include\_bd\_addr\_seg` 后，第三次 run 通过。

14. 板级验证结果

最新通过的 board run：

field	value
run_id	m7_board_20260502_083547
port	COM9
baud	115200
program_returncode	0
capture_returncode	0
case_count	3
failures	0
uart_log	C:\Users\AIALRA-PORTABLE\Desktop\Vertebi\viterbi_startup_pack\src\data\board_runs\m7_board_20260502_083547\uart.log

case	decoded_bits	cycles	mismatches	result
no_noise	96	802	0	PASS
single_bit_error	96	855	0	PASS
soft_awgn_2db	96	855	0	PASS

**表：M7 UART case summary。 ** 数据来源 `data/board\_runs/m7\_board\_20260502\_083547/uart\_capture.json` 和 `data/board\_runs/board\_summary.csv`。每个 case 都看到 204 input samples 和 96 emitted output samples， mismatch_count 为 0。

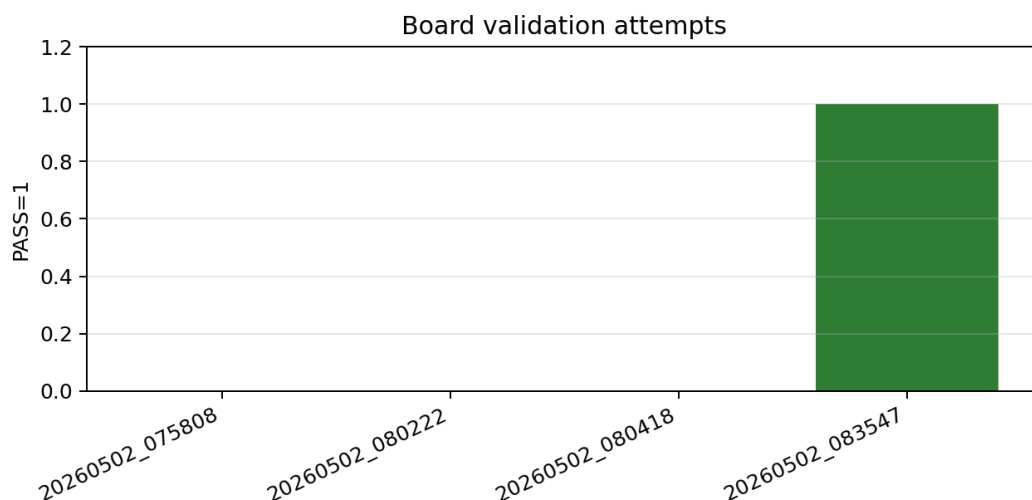


图 17：board validation attempts。 数据来源 `data/board\_runs/board\_summary.csv`。它保留失败尝试和最终 PASS，回答 M7 不是只贴成功日志，而是记录了真实调试路径。

15. 瓶颈分析与架构取舍

Dependency-bound（依赖受限：下一步 path metric 依赖上一步，像接力必须等上一棒交棒）是 Viterbi 的根本瓶颈。每个 trellis step 的 metrics 必须来自上一步 metrics。Compute-bound（计算受限：同一拍要做很多加法比较选择，像 64 个路口裁判同时开会）出现在 64-state ACS 更新。Capacity-bound（容量受限：状态、深度和 survivor memory 一起膨胀，像账本页数越来越多）限制 traceback 和多帧扩展。Bandwidth-bound（带宽受限：PS/PL 搬运和 soft symbol 宽度影响吞吐，像快递口太窄）在 board shell 中通过 AXI DMA 暴露。Accuracy-bound（精度受限：量化位宽决定信息保留，像尺子刻度太粗会量不准）体现在 soft width 和 path metric width。

本项目 hero 选择 3-bit soft、traceback depth 40、PM width

12、subtract-min。不是因为它理论上永远最好，而是因为当前数据闭环中它满足三个条件：M4 soft3 RTL 0 mismatch，M5 depth 40 当前向量 0 mismatch，M7 board 0 mismatch。代价是时序：M6 100 MHz hero implementation WNS -20.433 ns，critical path 来自一拍内 64-state soft ACS/reduction 的组合深度。未来要达到 100 MHz，需要 pipeline/retiming（流水线/重定时：把长组合路径切成多段，像把一次长跑拆成接力）。

16. 结论与未来工作

最终成果是一个可复现的端到端 Viterbi decoder 项目：模型、向量、RTL、仿真、Vivado、板级 UART 和最终报告都在仓库内留证。M7 最新 UART run 证明 no_noise、single_bit_error、soft_awgn_2db 在真实 ZU4EV board shell 上通过。项目没有伪造 timing 或 board 数据；M6 的 100 MHz 时序失败被保留为限制，M7 用 25 MHz board shell 完成功能验证。

未来工作有四项。第一，把 soft ACS 拆 pipeline，让 hero 设计在 100 MHz 目标下 timing clean。第二，扩展更长随机帧和更多 SNR 点，生成真正统计意义上的 BER 曲线。第三，修通 DDR DMA buffer，避免 OCM 容量限制。第四，把 survivor memory 改成更资源友好的 RAM 结构，减少 FF 压力。

17. 参考资料与工具声明

参考资料和事实源：

资料	用途
spec/viterbi_spec.json	设计参数唯一事实源
data/model/model_unit_test_summary.json	Python 测试结果
data/regression/rtl_regression_summary.csv	hard RTL regression
data/regression/soft3_regression_summary.csv	soft3 RTL regression
data/analysis/sweep_results.csv	M5 原始 sweep 数据
data/analysis/ber_summary.csv	由 M5 派生的有限向量 mismatch-rate 汇总
data/impl/vivado_summary.csv	M6 Vivado 资源、时序、功耗
data/board_runs/board_summary.csv	M7 UART board run 汇总
WORKLOG.md / DECISIONS.md / EXPERIMENTS.yaml	过程记录和决策依据

AI tools were used as engineering assistants for planning, code organization, script generation, debugging guidance, and report drafting. Final architectural decisions, validation criteria, experiment interpretation, and submission responsibility remain with the author.

中文解释：AI 工具用于辅助规划、代码组织、脚本生成、调试建议和报告草拟；最终架构决策、验证标准、实验解释和提交责任由作者承担。